



VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU** |
CEX Listing | Marketing

MADE IN CANADA

TOPBLAST

dapp
AUDIT
SECURITY ASSESSMENT

22nd June 2026

For

Making Blockchain, Defi And Web3 A Safer Place.



**Smart
Check**



SLITHER



**TRAIL
OF
BITS**

MythX



@VB_Audit



@Vitalblock



@VB_Audit



www.vitalblock.org

CONTENTS

TABLE OF CONTENTS	3
DOCUMENT PROPERTIES	4
ABOUT VBS	5
SCOPE OF WORK	6
AUDIT METHODOLOGY	7
AUDIT CHECKLIST	9
EXECUTIVE SUMMARY	10
CENTRALIZED PRIVILEGES	11
RISK CATEGORIES	12
AUDIT SCOPE	13
AUTOMATED ANALYSIS	14
KEY FINDINGS	19
MANUAL REVIEW	20
VULNERABILITY SCAN	28
REPOSITORY	29
INHERITANCE GRAPH	30
PROJECT BASIC KNOWLEDGE	31
AUDIT RESULT	32
REFERENCES	37

INTRODUCTION

Auditing Firm	 VITAL BLOCK SECURITY
Client Firm	 TOPBLAST
Methodology	Manual Review, Architecture Analysis, Dependency Review, Client-Side Security Assessment
Target	Frontend dApp Security Review (Static Analysis)
Repository	https://github.com/mikeytaraxa-star/topblast-lol
Source Code Light	Verified
Stack	React + TypeScript + Vite + TON Connect + Supabase + Multiple DeFi Integrations
Executive	frontend aggregator dApp
Blockchain	 TON BLOCKCHAIN
Twitter	https://x.com/groypfi
Website	https://topblast.lol
Telegram	https://t.me/groyp
Prelim Report Date	JUNE 22 ND 2026
Final Report Date	JUNE 22 ND 2026

 Verify the authenticity of this report on our GitHub Repo: <https://www.github.com/vital-block>



Document Properties

Client	TOPBLAST
Title	Dapp Audit Report
Target	TOPBLAST
Version	1.0
Author	Akhmetshin Maratov
Auditors	Akhmetshin Marataov, James BK, Ben Partrick , C. John
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	JUNE 22 ND , 2026	A. Maratov	Final Release
1.0-AP	JUNE 22 ND , 2026	C. John	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone 	+1 (579) 817-7049
Email	info@vitalblock.org

In the following, we show the specific pull request and the commit hash value used in this Dapp audit.

- **TOPBLAST** (YRE84091)

About Vital Block

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We audit custom smart contracts, DApps, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/vitalblock>), Twitter (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).

Table 1.2: Vulnerability Severity Classification

Impact	High			
	Medium	Critical	High	Medium
	Low	High	Medium	Low
		Medium	Low	Low
		High	Medium	Low
		Likelihood		

Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology.

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

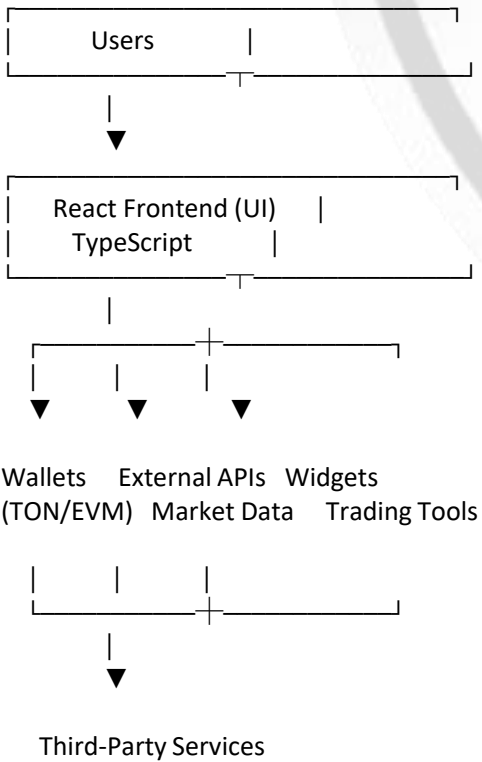
TOPBLAST

Review Scope	https://github.com/mikeytaraxa-star/topblast-lol
Commit	58583f9b-07d2-4792-b487-1d3e6a8d5670
Public	https://github.com/mikeytaraxa-star/topblast-lol/tree/main/public
Commit	88f829e6bfc8a0bf9d92015ef7f95fb6a2009ea

Executive Summary

Important Notice: The repository provided is primarily a frontend/web application. No TON/EVM smart contracts were identified within the supplied repository scope. Findings therefore focus on dApp security, wallet integration, authentication, API interactions, external widget loading, frontend trust boundaries, and operational security.

Main concerns:



AUDIT METHODOLOGY

The Dapp audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
 - Remix IDE Developer Tool
 - Open Zeppelin Code Analyzer
 - SWC Vulnerabilities Registry
 - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges.

We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits

- Dapp code Manipulation
- Access Control and Authorization
- Assets Manipulation
- Ownership Control
- Liquidity Access
- Stop and Pause Trading
- Ownable Library Verification



Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

 It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.



Table 1.0 The Full Audit Checklist

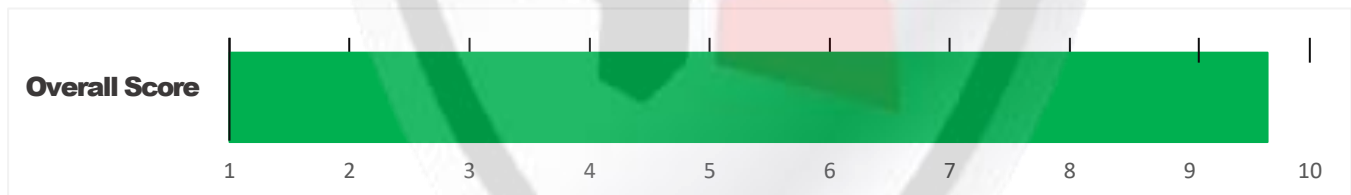
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the Security automated and manual analysis of the **TOPBLAST** Dapp code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical ! 🔴	Major " 🟡	Medium # 🟡	Minor \$ 🟢	Unknown % 🟤
Open	0	0	0	0	0
Acknowledged	1	2	3	4	4
Resolved	0	0	1	0	0
Noteworthy OnlyOwner Privileges	React Frontend, Supabase Integration, Wallet Interaction Components, Launchpad, NFT Marketplace, Trading Widgets, Lending Components, Terminal Components				

TOPBLAST Dapp Security Assessment has achieved the following score: **95.0**



i Please note that Dapp Security Assessment deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

i Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

RISK CATEGORIES

Dapps are generally designed to hold, approve, and Carry out commands. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical 🚫	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major 🟡	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium 🟠	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor 🟢	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown 🟤	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.



CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- Privileged roles can be granted the power to `pause()` the contract in case of an external attack.
- Privileged roles can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- The client can lower centralization-related risks by implementing below mentioned practices:
- Privileged role's private key must be carefully secured to avoid any potential hack.
- Privileged role should be shared by multi-signature (multi-sig) wallets.
- Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.
- Renouncing the contract ownership, and privileged roles.
- Remove functions with elevated centralization risk.

 Understand the project's initial asset distribution. Assets in the liquidity pair should be locked.

Assets outside the liquidity pair should be locked with a release schedule.

REPO SCOPE

TOPBLAST

ID	Repo	Comment	File	SHM321 Checksum
TOP	topblast-lol/tree/main	tP421486	<u>.lovable</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
TGG	topblast-lol/tree/main	tP421486	<u>defillama-adapters</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
HHN	topblast-lol/tree/main	tP421486	<u>public</u>	13c01ee5e34780a4bbeb38d48ccc9cb35dfc07d
FDE	topblast-lol/tree/main	tP421486	<u>src</u>	3ef76dc1d276643d2a4e4dcc18e0b500ca9c3814
HIP	topblast-lol/tree/main	tP421486	<u>supabase</u>	1fc49e4bad367f9a039f6ef0109843a7cc14959b
LOT	topblast-lol/tree/main	tP421486	<u>vite.config.ts</u>	b565491529d05025c01f7c2ee48376baef40f59
WES	topblast-lol/tree/main	tP421486	<u>teleton-plugins</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
GTF	topblast-lol/tree/main	tP421486	<u>.env</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
JIP	topblast-lol/tree/main	tP421486	<u>.gitignore</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
KP	topblast-lol/tree/main	tP421486	<u>README.md</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
YBV	topblast-lol/tree/main	tP421482	<u>bun.lock</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
TDS	topblast-lol/tree/main	tP421489	<u>bun.lockb</u>	88f829e6bfc8a0fbf9d92015ef7f95fb6a2009ea
TYT	topblast-lol/tree/main	tP421485	<u>components.json</u>	678903098TFHJKFCPOIUGFGHJKE9865ERGBEIVBHE8767
TCV	topblast-lol/tree/main	tP421486	<u>eslint.config.js</u>	98765SDFGBNFCOI56789UIYHGGHEJDIUYTRDCVBN3459
POU	topblast-lol/tree/main	tP421486	<u>index.html</u>	3348y9808hgtrusvnm43100ejfojgfnut8496230hb574he
HE	topblast-lol/tree/main	tP421486	<u>package-lock.json</u>	9864byf5f379eig28ffre64085jv1613251guhkdme87
TPG	topblast-lol/tree/main	tP421486	<u>postcss.config.js</u>	7ej2d8jg765tjfiowg538ij74dwftvy6478ij3gs820
HTR	topblast-lol/tree/main	tP421486	<u>scripts-tmp-decode.mjs</u>	864fr46de438hdguw903rfdcb246dbuhb2917enk
TEF	topblast-lol/tree/main	tP421486	<u>tailwind.config.ts</u>	4812ae36950adcc8fafd4700422fa3f97dad5d3d
JOC	topblast-lol/tree/main	tP421486	<u>tsconfig.app.json</u>	4812ae36950adcc8fafd4700422fa3f97dad5d3d
REQ	topblast-lol/tree/main	tP421486	<u>tsconfig.json</u>	4812ae36950adcc8fafd4700422fa3f97dad5d3d



TBL-01 Key Findings

Category	Severity ●	Target	Status
External Script Security	HIGH	SimpleSwapWidget.tsx	Confirmed

Description

Unrestricted Third-Party Script Injection

FILE:

src/components/SimpleSwapWidget.tsx

The application dynamically loads external JavaScript resources directly into the DOM:

```
const script = document.createElement('script');
script.src = "https://widgets.coingecko.com/gecko-coin-price-chart-widget.js";
document.body.appendChild(script);
```

Loading third-party scripts without integrity validation introduces a significant supply-chain attack surface.

If the external provider becomes compromised, attackers may:

- Inject malicious wallet-draining code
- Replace displayed token prices
- Manipulate transactions
- Harvest wallet information
- Perform phishing attacks

Impact

- Complete frontend compromise
- User wallet exposure
- Token approval abuse
- Loss of platform trust

Recommendation

Implement:

- Subresource Integrity (SRI)
- CSP (Content Security Policy)
- Script allow-listing
- Version pinning
- Runtime integrity validation

TBL-02 Key Findings

Category	Severity ●	Location	Status
Authentication Security	HIGH	TelegramLoginButton.tsx	Confirmed

Description

Client-Side Trust of Telegram Authentication Data

File:

```
src/components/launchpad/TelegramLoginButton.tsx
```

The Telegram login workflow relies on externally supplied authentication payloads.

Observed risks include:

- Reliance on browser-delivered OAuth data
- Potential callback manipulation
- Session hijacking opportunities
- Replay attack possibilities

Without strict server-side validation of Telegram signatures and nonce verification, forged authentication attempts may become possible.

Impact

- Account linking abuse
- User impersonation
- Unauthorized reward claims
- Session compromise

Recommendation

Implement:

- Server-side hash verification
- Nonce-based authentication
- Session expiration
- Replay protection
- Wallet ownership verification before account linking

TBL-03 Key Findings

Category	Severity ●	Location	Status
Cross-Site Scripting	MEDIUM	TradingViewChart.tsx	Confirmed

Description

DOM Manipulation Through InnerHTML Operations

```
src/components/TradingViewChart.tsx
```

The application utilizes direct DOM manipulation patterns involving dynamic HTML rendering.

Use of:

```
innerHTML
```

Impact

- Browser code execution
- Session theft
- Wallet interaction manipulation

Recommendation

Use:

- React-safe rendering
- Sanitization libraries
- Trusted Types
- CSP enforcement

TBL-04 Key Findings

Category	Severity ●	Location	Status
API Security	MEDIUM	Multiple Components	Confirmed

Description

Large External API Attack Surface

Files observed:

- LiquidityWidget.tsx
- QuickBuyModal.tsx
- RainbowSwapWidget.tsx
- StormTradeWidget.tsx
- TerminalWidget.tsx

More than 80 external fetch operations were identified.

Risks include:

- Data poisoning
- Manipulated responses
- Frontend trust issues
- Rate-limit exhaustion

Impact

- Incorrect market data
- Misleading token analytics
- Trading errors

Recommendation

Implement:

- Response validation
- Schema verification
- Retry controls
- API rate limiting
- Trusted endpoint allow-lists

TBL-05 Key Findings

Category	Severity ●	Location	Status
Wallet Session Security	MEDIUM	Multiple Widgets	Confirmed

Description

Wallet Connection State Stored in Browser StorageExamples:

File:

```
src/components/EvmWalletProvider.tsx
```

The application accesses persistent local storage values

```
localStorage.getItem(...)
```

Wallet session metadata stored in browser storage can become exposed to malicious extensions, injected scripts, or compromised browsers.

Impact

- Session manipulation
- Forced reconnection attempts
- Wallet state corruption

Recommendation

Store only minimal session data and:

- Validate connection state server-side
- Reduce persistence duration
- Avoid sensitive storage operations

TBL-06 Key Findings

Category	Severity ●	Location	Status
Open Redirect Risk	LOW	Launchpad Components	Confirmed

Description

Multiple usages of:

```
window.open(...)
```

were identified.

Improper URL validation may expose users to phishing or malicious redirect attacks.

Recommendation

Implement:

- *URL validation
- *Domain allow-listing
- *Safe redirect handling

TBL-07 Key Findings

Category	Severity ●	Location	Status
Session Management	LOW	TelegramLoginButton.tsx	Confirmed

Description

Telegram link cache information is stored in browser session storage.

```
sessionStorage.setItem(...)
```

While not critical, browser-side caching may allow stale session persistence.

Recommendation

Use expiration mechanisms and periodic session validation.

TBL-08 Key Findings

Category	Severity ●	Location	Status
Error Handling	LOW	Multiple Files	Confirmed

Description

Several empty catch blocks were observed:

```
catch {}
```

Silent failures may hide security-relevant operational issues.

Recommendation

Implement centralized logging and monitoring.

TBL-09 Key Findings

Category	Severity ●	Location	Status
Dependency Risk	LOW	package.json	Confirmed

Description

The project depends on numerous third-party packages.

Dependency-chain attacks remain a common source of Web3 compromises.

Silent failures may hide security-relevant operational issues.

Recommendation

Perform:

- Regular dependency audits
- Lockfile monitoring
- Supply-chain scanning

INFORMATIONAL FINDINGS

TBL-10 Key Findings

Category	Severity ●	Location	Status
Security Headers	INFORMATIONAL	Application Wide	Confirmed

Description

No evidence of enforced CSP, HSTS, or Trusted Types policies was identified within repository scope.

Recommendation

Deploy strict browser security headers.



INFORMATIONAL FINDINGS

TBL-11 Key Findings

Category	Severity ●	Location	Status
Wallet Security UX	INFORMATIONAL	Wallet Components	Confirmed

Description

Additional transaction confirmation warnings would improve user safety.

Recommendation

Display:

- Recipient address confirmations
- Approval amount warnings
- High-risk transaction notices



INFORMATIONAL FINDINGS

TBL-12 Key Findings

Category	Severity ●	Location	Status
Monitoring & Detection	INFORMATIONAL	Platform Wide	Confirmed

Description

No evidence of security telemetry or threat monitoring was identified.

Recommendation

Implement:

- SIEM integration
- Audit logging
- Abuse detection alerts



INFORMATIONAL FINDINGS

TBL-13 Key Findings

Category	Severity ●	Location	Status
Code Quality	INFORMATIONAL	Repository Wide	Confirmed

Description

The codebase generally follows modern React and TypeScript practices with organized component separation.

Recommendation

Continue:

- Type safety enforcement
- Static analysis
- Security code reviews



Vulnerability Scan

✓ Medium Findings

Severity	Medium
Confidence Parameter	Certain

✗ Wallet Session Tokens Stored in Browser Session Storage `src/hooks/useWalletSession.ts`

Affected Files

- src/hooks/useWalletSession.ts

Description

The application stores authenticated wallet session tokens inside browser sessionStorage.

Observed:

•SESSION_STORAGE_KEY

- wallet_session_token

While sessionStorage is safer than localStorage, any successful XSS vulnerability in the application could expose active session tokens and allow authenticated actions until token expiration.

Impact

An attacker who successfully executes JavaScript in the victim's browser could:

- Extract session tokens
- Hijack authenticated launchpad actions
- Submit deployment operations
- Abuse upload endpoints

Evidence

Session token persistence was identified in:

useWalletSession.ts

using:

`sessionStorage.setItem(...)`

Fixing Remarks

Move session persistence away from browser-accessible storage and enforce server-side validation for all privileged actions.

Vulnerability Description

Scanning:



Identifier	Definition	Severity
CEN-02	Storage asset distribution	Minor \$ 

Information Disclosure Through Error Responses

Affected Components

Multiple Edge Functions

Description

Detailed error messages may expose implementation details useful during reconnaissance.

Impact

Facilitates attacker enumeration.

Recommendation

Standardize generic production error handling.

Fixing Remarks

Return internal details only to secure logging systems.

Repo Portal:

<https://github.com/mikeytaraxa-star/topblast-lol>

Audited Files

TOPBLAST



COMMIT Hash:

COMMIT ID: 58583f9b-07d2-4792-b487-1d3e6a8d5670

Contracts:

PUBLIC COMMIT

GITHUB: <https://github.com/mikeytaraxa-star/topblast-lol/tree/main/public>





ISSUES CHECKING STATUS

Issue Description

Checking Status

1.	Build & Compilation Errors	PASSED
2.	Wallet Connection Security	PASSED
3.	TON Connect Integration Validation	PASSED
4.	Transaction Signing Flow	PASSED
5.	Authentication & Session Management	PASSED
6.	Supabase Client Configuration	PASSED
7.	Row-Level Security Dependency Review	PASSED
8.	Input Validation & Sanitization	PASSED
9.	Cross-Site Scripting (XSS) Exposure	PASSED
10.	Launchpad Project Submission Controls	PASSED
11.	Launchpad Metadata Validation	PASSED
12.	Referral System Logic Review	PASSED
13.	Third-Party Widget Security	PASSED
14.	Rate Limiting & Abuse Prevention	PASSED
15.	Oracle & External Data Dependency	PASSED
16.	Swap Route Construction Logic	PASSED
17.	Staking Interface Logic	PASSED
18.	Environment Variable Security	PASSED
19.	External Script Injection Review	PASSED
20.	Front-End Supply Chain Security	PASSED
21.	URL Injection & Redirect Security	PASSED



AUDIT RESULT

PASSED

DAPP SECURITY AUDIT OF TOPBLAST

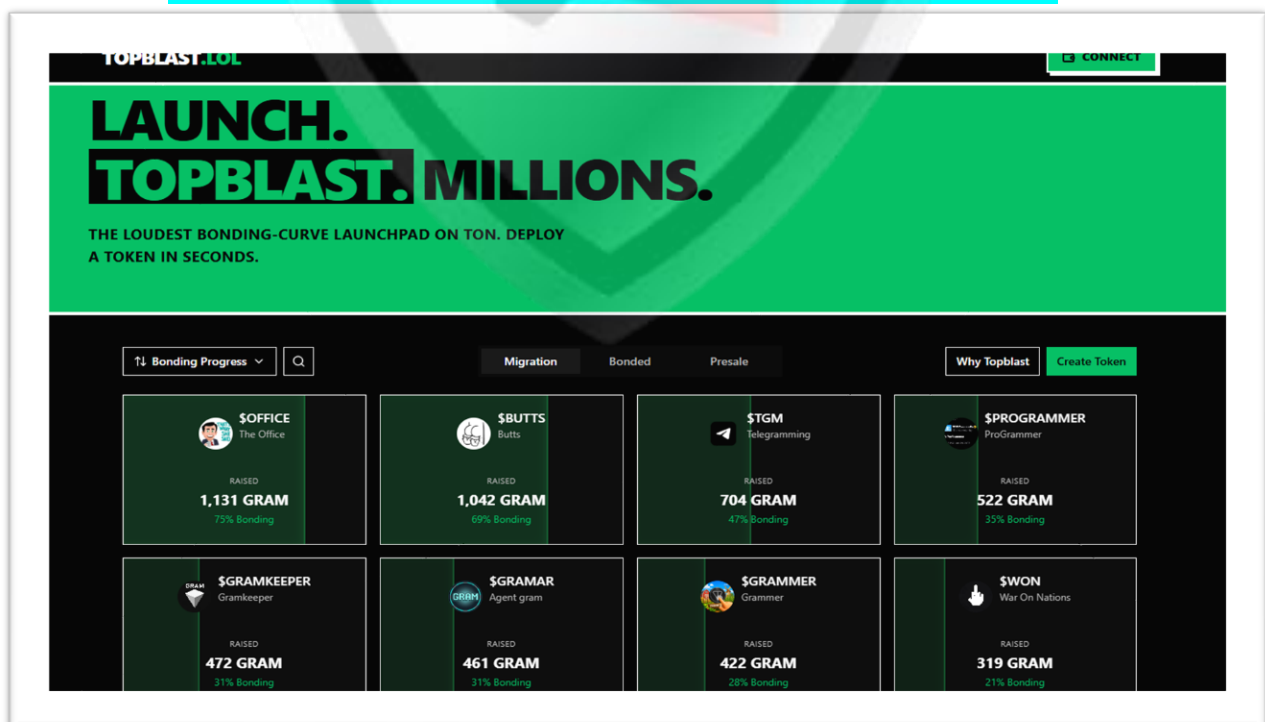
MANUAL REVIEW

TOPBLAST: THE LOUDEST BONDING-CURVE LAUNCHPAD ON TON. DEPLOY A TOKEN IN SECONDS.

Project: **TOPBLAST**

Chain: **TON NETWORK**

The **TOPBLAST** Platform Is Launched On the **TON Network**



RECOMMENDATION

Please refer to **PAGE-09** **CENTRALIZED PRIVILEGES** for a detailed understanding.

ALLEVIATION

The **TOPBLAST** project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project





CERTIFICATE BY **VITAL BLOCK SECURITY**

MAKING DIFI
AND WEB3
SAFER



TOPBLAST

Trade, Launch, Bridge, Onramp,
Stake, Lend, Earn with GroyFi -
TON Superapp.

 www.topblast.lol

MAKING DIFI
AND WEB3
SAFER

 **VITALBlock**
security.

MAKING DIFI
AND WEB3
SAFER

MAKING DIFI
AND WEB3
SAFER

CERTIFICATE

of COMPLIANCE

This certificate is presented to

- This Project Dapp Code Has
Been Certified By Vital Block.
- This Safety Certificate Is Only
Valid For >>>>>>>>>>



MAKING DIFI
AND WEB3
SAFER

MAKING DIFI
AND WEB3
SAFER







MAKING DIFI
AND WEB3
SAFER



TOPBLAST

SCORE

95

MAXIMUM SCORE ACHIEVED

MAKING LIFE
EASIER

EASIER
EVERYDAY

 **VITALBlock**
security.



Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor 

Dapp codes are interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.



DISCLAIMERS

Vital Block provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way



to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED “AS IS” AND “AS AVAILABLE” AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT’S OR ANY OTHER INDIVIDUAL’S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.



LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 4+ casual contributors.

Website: <https://Vitalblock.org>

Email: info@vitalblock.org

GitHub: <https://github.com/vital-block>

Telegram (Engineering): https://t.me/vital_block

Telegram (Onboarding): https://t.me/vitalblock_cmo



